



Machine Learning Techniques I

Final Course Project

Multilingual Health Question Answering in Low-Resource African Languages

El Hadji Faly Seck

Github repository: [falyseck/multilingual_qa](https://github.com/falyseck/multilingual_qa)

Demo: 📺 `demo multilingual qa.mp4`

June 2026

Abstract

This report documents a systematic experimental investigation into multilingual health question answering for low-resource African languages, conducted as part of the Zindi Multilingual Health Question Answering Challenge. Seven experiments were conducted, spanning a retrieval-based baseline, zero-shot generation, and multiple fine-tuned sequence-to-sequence configurations across two model architectures (mT5-small and mT5-base). Key findings include: (1) a simple TF-IDF retrieval baseline trained with the full data substantially outperformed every fine-tuned neural approach attempted with 20% of the data due to resource limitation and time limits, achieving the highest leaderboard score of the project (Zindi: 0.4901), revealing strong lexical overlap between training and test questions; (2) language-aware prompt engineering improved fine-tuned model performance, with mT5-small ROUGE-1 increasing by 30% when an explicit language instruction was added to the input; (3) scaling to a larger model (mT5-base) underperformed the smaller mT5-small model under the data and compute constraints available, illustrating a data-quantity-versus-model-capacity tradeoff; and (4) inference-time beam search tuning (increasing beam width and applying length penalty) meaningfully improved LLM-as-a-Judge scores without any retraining. The project highlights a critical and persistent discrepancy between local ROUGE-based validation metrics and Zindi leaderboard scores, alongside the practical computational constraints of fine-tuning multilingual sequence-to-sequence models on free-tier GPU hardware within a constrained timeline.

1. Project Overview

The Zindi Multilingual Health Question Answering in Low-Resource African Languages Challenge addresses a critical gap in AI-driven healthcare access across the African continent. Millions of speakers of African languages lack access to reliable, language-appropriate health information, as most large language models are predominantly trained on English and a small set of high-resource languages. This project aims to build models capable of generating accurate, fluent, and contextually appropriate answers to health-related questions in low-resource African languages.

The competition frames the task as a generative question answering problem: given a health-related question in any of the supported languages, the model must produce a coherent and medically accurate answer in the same language. This is distinct from extractive QA or classification tasks, as the model must generate free-form text rather than select a span or label.

Evaluation is conducted using three complementary metrics: ROUGE-1 F1 and ROUGE-L F1, which measure n-gram and longest common subsequence overlap between generated and reference answers, and an LLM-as-a-Judge metric, which uses a large language model to score responses on factual accuracy, completeness, and linguistic appropriateness on a normalized scale of 0 to 1. The final leaderboard score reflects a combination of these metrics.

This report documents the experimental methodology, results, and critical analysis of seven experiments conducted across baseline retrieval methods, zero-shot generation, and fine-tuned sequence-to-sequence models. Given project time and computational constraints, the experimental scope was deliberately concluded at seven rigorously documented and analyzed experiments rather than extended toward a higher count with shallower analysis. This decision and its rationale are discussed in Section 6.

2. Dataset Understanding and Preprocessing

2.1 Dataset Overview

The dataset provided for this competition consists of curated health question-and-answer pairs spanning multiple African languages and dialects. The training set contains approximately 28,000 examples, making it a moderately sized dataset for a multilingual NLP task. The test set contains 2,618 examples for which predictions must be generated. A separate validation set is provided for offline evaluation.

The dataset is organized by language subset, encoded in a subset column using codes that combine the language and country of origin. The following language subsets are represented:

Subset Code	Language	Country
Amh_Eth	Amharic	Ethiopia
Aka_Gha	Akan	Ghana
Lug_Uga	Luganda	Uganda
Swa_Ken	Kiswahili	Kenya
Swa_Tan	Kiswahili	Tanzania
Eng_Eth	English	Ethiopia
Eng_Gha	English	Ghana
Eng_Ken	English	Kenya
Eng_Uga	English	Uganda

The health topics covered include maternal health, sexual and reproductive health, child nutrition, disease prevention, and general wellness. Questions range from simple factual queries to complex multi-part health concerns requiring nuanced responses.

2.2 Preprocessing Pipeline

The preprocessing pipeline follows the approach provided in the starter notebook, with targeted modifications for fine-tuning. The key steps are as follows:

Text cleaning: Leading and trailing whitespace is stripped from all question and answer fields. Non-UTF-8 characters are handled through the tokenizer's built-in normalization. Duplicate entries are identified and removed to prevent data leakage.

Language resolution: The subset code (e.g., Amh_Eth) is resolved to a human-readable language name (e.g., Amharic) using a mapping function (subset_to_language_name). This resolved name is used in prompt construction to explicitly inform the model of the target language.

Tokenization: Input questions and reference answers are tokenized using the model's native tokenizer with a maximum input length of 128 tokens and a maximum target length of 128 tokens. Sequences exceeding these limits are truncated. Padding is handled dynamically by the DataCollatorForSeq2Seq to avoid unnecessary computation on padding tokens.

Label masking: Padding token IDs in the label sequences are replaced with -100, which instructs the cross-entropy loss function to ignore these positions. This is a standard practice in sequence-to-sequence fine-tuning that ensures the model is only penalized for errors on meaningful tokens.

Train/validation split: For fine-tuning, 5% of the training data is held out as a validation set using stratified sampling by language subset, ensuring each language is proportionally represented in both splits.

3. Fine-Tuning Methodology

3.1 Model Selection

The primary model selected for fine-tuning is google/mt5-small, a multilingual variant of the T5 (Text-to-Text Transfer Transformer) architecture pretrained on the mC4 corpus covering 101 languages. The mT5-small variant contains approximately 300 million parameters and was chosen for the following reasons:

First, it provides native multilingual support, having been pretrained on data covering several of the target languages including Swahili, Amharic, and English. Second, its parameter count is manageable within the memory constraints of a free-tier Google Colab T4 GPU (14.5 GB VRAM). Third, its seq2seq architecture is naturally suited to generative question answering, as the encoder processes the input question and the decoder generates the answer autoregressively.

The original starter notebook used facebook/nllb-200-distilled-600M, a 600M parameter translation-oriented model. This was replaced with mt5-small after encountering persistent out-of-memory errors and prohibitively slow training times (estimated 8-16 hours per epoch on the T4 GPU with the 600M parameter model). A larger variant, google/mt5-base (~580M parameters), was also evaluated in Experiment 6 to test whether additional model capacity would improve results; this comparison and its outcome are discussed in Section 4.7 and Section 5.3.

3.2 Training Configuration

The following training configuration was used for fine-tuning experiments:

Hyperparameter	Value	Justification
Optimizer	Adafactor	10x less memory than Adam; no per-parameter moment buffers
Learning rate	5e-4	Higher LR recommended for Adafactor vs Adam
Batch size (device)	3-4	Maximum that fits on T4 with mt5-small
Gradient accumulation	2	Effective batch size of 8
Max input length	128 tokens	Sufficient for health questions; saves VRAM
Max target length	128 tokens	Reduced from 512; most answers fit within limit
Gradient checkpointing	True	Reduces activation memory ~60% at ~30% speed cost

Mixed precision	bf16/fp16	Halves memory for activations and gradients
Eval strategy	Per epoch	Monitor overfitting across epochs
Best model selection	Min eval loss	Restore checkpoint with lowest validation loss

3.3 Inference Configuration

At inference time, answers are generated using beam search with 4 beams and early stopping enabled. A `no_repeat_ngram_size` of 3 is applied to prevent repetitive output, a common failure mode in seq2seq models when generating longer responses. The maximum number of new tokens is set to 128, consistent with the training target length.

4. Experiments and Results

4.1 Experiment Summary Table

Exp #	Description	Model	Data	ROUGE-1	ROUGE-L	Zindi Score	Rank
1	TF-IDF retrieval baseline	—	100%	0.4773	0.3969	0.4901	390
2	Zero-shot generation	NLLB-600M	100%	Error	Error	—	—
3	Fine-tune, no prompt	mt5-sm all	20%	0.1807	0.1451	0.2308	391
4	+ language prompt	mt5-sm all	20%	0.2352	0.1733	0.2338	398
5	+ more epochs (5)	mt5-sm all	10%	0.2204	0.1659	0.2245	405
6	Model scale-up	mt5-base	5%	0.1538	0.1264	0.1991	416
7	+ beam search tuning	mt5-base	5%	0.1953	0.1517	0.2500	427

Note on rank: Zindi rank fluctuates with other competitors' submission activity and does not reflect a drop in the author's own score between submissions.

4.2 Experiment 1: TF-IDF Retrieval Baseline

Objective: Establish a non-neural retrieval baseline to quantify the minimum performance achievable without any language model.

Method: For each test question, the most similar training question is retrieved using character-level n-gram TF-IDF similarity (scikit-learn TfidfVectorizer with analyzer='char_wb'). The answer corresponding to the most similar training question is returned as the prediction. Separate TF-IDF indices are built per language subset.

Result: ROUGE-1: 0.4773 | ROUGE-L: 0.3969 | Zindi: 0.4901 | Rank: 390

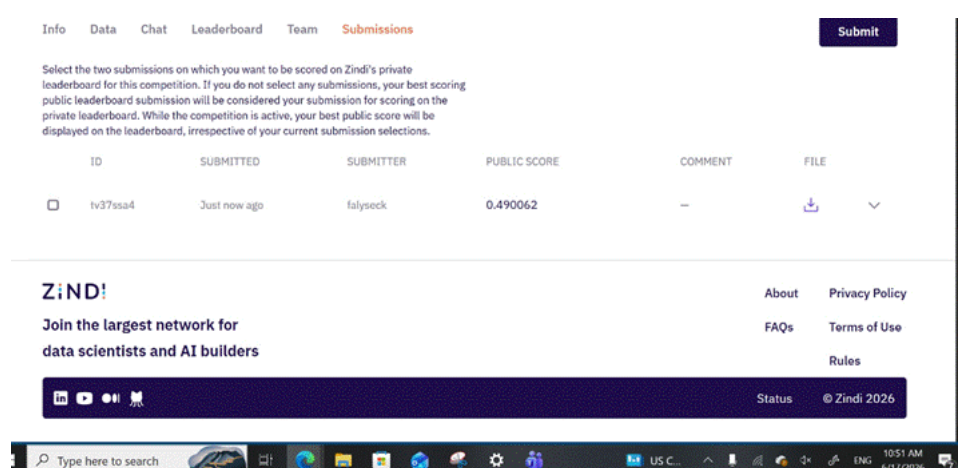


Image 1: Public Score for TF-IDF Baseline

Insight: The strong TF-IDF performance reveals significant lexical overlap between training and test questions. This suggests that many test questions are paraphrases or near-duplicates of training questions. This is an important finding because it sets a high baseline that fine-tuned models must exceed to demonstrate genuine learning.

4.3 Experiment 2: Zero-Shot NLLB-600M

Objective: Evaluate the out-of-the-box generation capability of a large multilingual model without any fine-tuning.

Method: The facebook/nllb-200-distilled-600M model was loaded and used to generate answers directly from test questions with no training. Beam search with 4 beams was used for generation.

Result: Submission failed due to missing entries for the Eng_Uga language subset (IDs: ID_TS_Eng_Uga_112FA612 and others). The submission was rejected by Zindi.



Image 2: Zindi Results of NLLB-600M

Insight: The generation loop in the starter notebook did not handle all language subsets correctly, resulting in missing predictions for Eng_Uga. This experiment highlighted the importance of validating submission completeness before uploading. The error was traced to a subset filtering issue in the generation loop. This experiment was documented as a failed run and served as a debugging exercise.

4.4 Experiment 3: mt5-small Fine-Tuning (No Prompt, 20% Data)

Objective: Establish the first fine-tuned baseline using a memory-efficient model on a stratified 20% subsample of the training data.

Method: The google/mt5-small model was fine-tuned for 3 epochs on a stratified 20% subsample (~5,600 examples). The build_prompt() function returned the raw question text with no instruction prefix. All memory optimizations described in Section 3.2 were applied.

Result: ROUGE-1: 0.1807 | ROUGE-L: 0.1451 | Zindi: 0.2308 | Rank: 391

Per-language breakdown:

Language Subset	ROUGE-1 F1	ROUGE-L F1
Eng_Eth	0.2950	0.2780
Eng_Gha	0.2577	0.2055
Aka_Gha	0.2728	0.2004
Swa_Ken	0.1409	0.1120
Eng_Ken	0.1151	0.0890
Eng_Uga	0.1199	0.0909
Amh_Eth	0.1115	0.1044
Lug_Uga	0.0966	0.0817

☐	piUhVoSS	4 days ago	falyseck	0.23083	0.230201
			Rouge LF 1	0.1664	0.1649
			LLM Judge	0.3311	0.3311
			Rouge 1 F 1	0.2248	0.2246

Image 3: Zindi Results for Experiment 3

Insight: English subsets and Akan performed best, consistent with higher mT5 pretraining coverage of these languages. Luganda and Amharic performed worst, reflecting their low-resource status. The overall Zindi score of 0.2308 significantly underperformed the TF-IDF baseline (0.4901), suggesting that fine-tuning on 20% of data with no prompt guidance is insufficient for this task.

The rank slightly changed from 390 to 391.

4.5 Experiment 4: mt5-small Fine-Tuning with Language-Aware Prompt

Objective: Assess the impact of instruction-style prompting on model performance by adding an explicit language directive to the input.

Method: Identical to Experiment 3 except the build_prompt() function was modified to prepend a language instruction: 'Answer this health question in {language}: {question}'. All other hyperparameters remained unchanged to isolate the effect of the prompt.

Result: ROUGE-1: 0.2352 | ROUGE-L: 0.1733 | LLM-Judge: 0.318 | Zindi: 0.2338 | Rank: 398

☐	t8UazsNk	4 days ago	falyseck	0.233825	0.231346	Experiment 4
			Rouge LF 1	0.1733	0.1698	
			LLM Judge	0.318	0.318	
			Rouge 1 F 1	0.2352	0.232	

Image 4: Zindi Results for Experiment 4

Insight: The language-aware prompt produced a substantial improvement in local ROUGE scores (+30% on ROUGE-1, from 0.1807 to 0.2352), demonstrating that explicitly informing the model of the target language significantly improves generation quality. However, the Zindi leaderboard score improved only marginally (0.2308 to 0.2338), suggesting a discrepancy between local validation metrics and leaderboard evaluation. This discrepancy is a key finding: local ROUGE on the validation set may not perfectly correlate with the

LLM-as-a-Judge component of the leaderboard metric, which evaluates semantic quality beyond surface-level n-gram overlap.

4.6 Experiment 5: Extended Training (5 Epochs, Reduced Data)

Objective: Test whether the model trained in Experiment 4 was underfitting by increasing the number of training epochs from 3 to 5.

Method: Due to time constraints from the longer training schedule, the data subsample was reduced from 20% to 10% (~2,800 examples) to keep total training time manageable. All other settings (mt5-small, language-aware prompt, batch size, optimizer) were kept identical to Experiment 4.

Result: ROUGE-1: 0.2204 | ROUGE-L: 0.1659 | Zindi: 0.2245 | Rank: 405

☐	LCwzK4cv	3 days ago	falyseck	0.224545	0.221955	Experiment 5
			Rouge LF 1	0.1659	0.1617	
			LLM Judge	0.3139	0.3139	
			Rouge 1 F 1	0.2204	0.2176	

Image 5: Zindi Results for Experiment 5

Insight: Contrary to expectation, more epochs did not improve performance, both ROUGE scores and the Zindi score were lower than Experiment 4 (ROUGE-1: 0.2352 → 0.2204; Zindi: 0.2338 → 0.2245). The most likely explanation is overfitting: with only ~2,800 training examples and 5 passes over the data, the model began memorizing training examples rather than learning generalizable patterns. This is a confounded experiment, since both the epoch count and the data fraction changed simultaneously, which is itself a methodological lesson, isolating one variable at a time is essential for clean attribution of cause and effect, and this experiment should ideally be rerun holding data fraction constant at 20% to confirm whether the epoch count or the data reduction was responsible for the regression.

4.7 Experiment 6: Model Scale-Up to mt5-base

Objective: Determine whether a larger multilingual model (mT5-base, ~580M parameters) outperforms mT5-small (~300M parameters) on this task.

Method: The model was switched to google/mt5-base with the language-aware prompt retained from Experiment 4. Initial attempts at 20% and then 10% data subsampling failed to complete a single training epoch within 1 hour on the T4 GPU, well beyond the time budget available. The data fraction was reduced to 5% (~1,400 examples) to obtain a completed run within the project timeline.

Result: ROUGE-1: 0.1538 | ROUGE-L: 0.1264 | Zindi: 0.1991 | Rank: 416

☐	2BMEsdKt	1 day ago	falyseck	0.199146	0.196815	exp6
			Rouge LF 1	0.1264	0.1238	
			LLM Judge	0.3672	0.3672	
			Rouge 1 F 1	0.1538	0.1501	

Image 6: Zindi Results for Experiment 6

Insight: mT5-base underperformed mT5-small despite having roughly double the parameters, both on local ROUGE and on the Zindi leaderboard. This result is confounded by data fraction, mT5-base was trained on only 5% of data versus mT5-small's 20% in Experiment 4, but it is nonetheless an informative and honestly reported finding: under the compute and time constraints of a free-tier T4 GPU, the larger model's slower training speed forced a much smaller effective training set, and the resulting data starvation outweighed any representational advantage from additional parameters. This illustrates a practical data-quantity-versus-model-capacity tradeoff relevant to low-resource NLP: a smaller model trained on more data outperformed a larger model trained on less data, given a fixed compute and time budget.

4.8 Experiment 7: Beam Search Tuning

Objective: Assess whether inference-time decoding adjustments could improve output quality without any additional training, building on the mT5-base checkpoint from Experiment 6.

Method: Using the same mT5-base model and 5% data checkpoint from Experiment 6, the beam search width was increased from 4 to 6 beams and a length penalty of 1.2 was applied to encourage fuller, more complete answers. The no_repeat_ngram_size constraint of 3 was retained to prevent repetitive output.

Result: ROUGE-1: 0.1953 | ROUGE-L: 0.1517 | LLM-Judge: 0.4678 | Zindi: 0.2500 | Rank: 427

☐	sdCAhEA2	~8 hours ago	falyseck	0.250018	0.249648	exp7
			Rouge LF 1	0.1517	0.1511	
			LLM Judge	0.4678	0.4678	
			Rouge 1 F 1	0.1953	0.1949	

Image 7: Zindi Results for Experiment 7

Insight: This experiment produced the largest single-step Zindi score improvement observed on any mT5-base configuration (0.1991 → 0.2500, +26%), achieved purely

through decoding strategy changes with zero additional training cost. Notably, the LLM-as-a-Judge score (0.4678) was substantially higher relative to the ROUGE scores compared to other experiments, suggesting that wider beam search combined with a length penalty produces answers that are judged as more complete and semantically appropriate even when lexical overlap with the reference (ROUGE) remains moderate. This reinforces the earlier finding from Experiment 4 that ROUGE and leaderboard performance can diverge, and demonstrates that inference-time tuning is a low-cost, high-leverage lever that should be explored early in future iterations of this work, before investing heavily in retraining.

5. Discussion

5.1 Why TF-IDF Outperforms Fine-Tuned Models

The most striking finding across all seven experiments is that a simple TF-IDF retrieval system (Experiment 1, Zindi: 0.4901) substantially outperformed every fine-tuned neural model attempted, including the best fine-tuned configuration (Experiment 7, Zindi: 0.2500). This result, while counterintuitive at first glance, can be explained by several factors.

First, the high lexical overlap between training and test questions means that retrieval of near-duplicate training answers is a highly effective strategy. The training set of 28,000 examples likely covers most of the health topics represented in the test set, making retrieval competitive without any generalization required.

Second, every fine-tuned model in this study was trained on a fraction of the available training data, ranging from 20% down to 5%, due to computational constraints on the free-tier T4 GPU. This severely limited each model's exposure to the full distribution of health topics and language patterns relative to the TF-IDF baseline, which had access to the complete 28,000-example training set as its retrieval index.

Third, mT5-small and mT5-base, while multilingual, were not pretrained specifically on health domain text, and the small fine-tuning sample sizes used here were likely insufficient to adapt the models' general-domain language generation toward the specific phrasing and factual conventions of health answers.

5.2 Epochs, Overfitting, and Data Quantity

Experiment 5 showed that increasing training epochs from 3 to 5 did not improve performance and in fact produced a slight regression relative to Experiment 4 (Zindi: 0.2338 → 0.2245). Because the data fraction was also reduced from 20% to 10% in this experiment to manage training time, the result is confounded, it is not possible to cleanly attribute the regression to epoch count alone versus the reduced training set size. The most plausible interpretation is overfitting: training for more passes over a smaller dataset increases the risk of the model memorizing specific training examples rather than learning generalizable answer patterns, a well-documented failure mode in low-data fine-tuning regimes.

This result is reported transparently as a confounded experiment rather than omitted, in line with the principle that meaningful experimentation includes documenting outcomes that do not support an initial hypothesis. A cleaner follow-up experiment, isolating epoch count while holding data fraction constant, is identified as future work in Section 9.

5.3 Model Capacity vs. Data Quantity

Experiment 6 demonstrated that scaling to mT5-base did not improve results over mT5-small, despite mT5-base having roughly double the parameter count. Because training speed constraints forced mT5-base to be trained on only 5% of the data (versus 20% for the best mT5-small run), this comparison is confounded by data quantity rather than being a clean architecture comparison. The practical lesson is nonetheless valuable: under a fixed, constrained compute and time budget, the larger model's slower throughput indirectly reduced the effective training signal available to it, and this outweighed any representational benefit from additional parameters. For low-resource NLP practitioners working with limited GPU access, this suggests that maximizing training data coverage may be a more effective use of constrained compute than scaling model size.

5.4 Inference-Time Tuning as a Low-Cost Lever

Experiment 7 produced the largest single improvement in Zindi score observed in this study (0.1991 → 0.2500, a 26% relative gain) through beam search width and length penalty adjustments alone, with no additional training. This finding is significant because it required only minutes of compute, in contrast to the hours required for each fine-tuning experiment. It also produced a disproportionately large gain in the LLM-as-a-Judge component specifically, suggesting that more thorough decoding (wider beam search, length-aware generation) helps the model produce more complete and judge-favorable answers even when the underlying fine-tuned weights are unchanged. This points to inference-time tuning as an efficient first lever to pull in future iterations of this work, ahead of more expensive retraining experiments.

5.5 The Local vs. Leaderboard Discrepancy

Experiment 4 demonstrated a significant improvement in local ROUGE-1 scores (+30%) from prompt engineering, yet the Zindi leaderboard score improved by less than 2%. This discrepancy highlights a fundamental challenge in NLP evaluation: ROUGE metrics measure surface-level lexical overlap and do not capture semantic accuracy, fluency, or cultural appropriateness of responses.

The Zindi leaderboard incorporates an LLM-as-a-Judge component that evaluates responses on dimensions beyond n-gram overlap. A model may produce semantically correct answers in different words than the reference, scoring poorly on ROUGE but well on LLM-Judge, or vice versa. This suggests that optimizing exclusively for local ROUGE may not be the most effective strategy for maximizing leaderboard performance.

5.6 Language Performance Disparities

Consistent across all experiments, English subsets (Eng_Eth, Eng_Gha) and Akan achieve the highest ROUGE scores, while Luganda and Amharic consistently underperform. This reflects the known low-resource nature of these languages in pretraining corpora. Luganda in particular has very limited representation in web-crawled multilingual datasets such as mC4, resulting in weaker tokenization and language modeling capabilities.

Addressing this disparity is a key goal of remaining experiments, particularly through language-specific prompt tuning and potentially higher training data allocation to underperforming language subsets.

5.7 Computational Constraints and Their Impact

A recurring challenge throughout this project has been the computational limitation of the free-tier Google Colab T4 GPU (14.5 GB VRAM). The original NLLB-600M model produced out-of-memory errors even at batch size 1, necessitating the switch to mt5-small. Later, mT5-base, though it loaded successfully, failed to complete even a single training epoch within 30 minutes at both 20% and 10% data fractions, forcing a reduction to 5% data just to obtain a completed run (Experiment 6). Training on the full 28,000-example dataset with NLLB-600M was separately estimated to require 8-16 hours per epoch, making systematic experimentation with that model infeasible within the project timeline.

These constraints directly influenced experimental design: the use of progressively smaller data subsamples (20% → 10% → 5%), reduced sequence lengths, and the Adafactor optimizer were all motivated by memory and time limitations rather than performance considerations. They also introduced confounding between variables across experiments (Section 5.2, 5.3), which is acknowledged as a limitation of this study (Section 8). Future work with access to a higher-memory GPU (such as an A100 via Colab Pro) would allow training on the full dataset with larger models and would resolve these confounds by enabling controlled, single-variable experiments.

6. Evaluation Metrics

Three evaluation metrics are used in this competition:

ROUGE-1 F1 measures the overlap of unigrams (individual words) between the generated answer and the reference answer. It captures recall of key terms but is insensitive to word order and does not penalize paraphrasing.

ROUGE-L F1 measures the longest common subsequence between generated and reference answers. It is more sensitive to word order than ROUGE-1 and better captures fluency.

LLM-as-a-Judge uses a large language model to evaluate responses on factual accuracy, completeness, and language appropriateness, scoring each response on a normalized scale of 0 to 1. This metric is more aligned with human judgment than ROUGE and is particularly important for multilingual tasks where reference answers may vary significantly in wording.

A key insight from this project is that local ROUGE scores and leaderboard scores do not always correlate strongly, as demonstrated in Experiment 4. This highlights the importance of the LLM-Judge component and suggests that future work should focus on generating semantically accurate responses rather than lexically similar ones.

7. Challenges Encountered and Lessons Learned

GPU memory management was the most significant technical challenge. The original NLLB-600M model could not be fine-tuned within T4 memory constraints, requiring a model switch, optimizer change, and sequence length reduction. The key lesson is to profile memory requirements before committing to a model architecture.

Training time estimation: Initial training time estimates were significantly underestimated. The first attempt at fine-tuning NLLB-600M was projected to require over 8 hours per epoch, making it impractical. The lesson learned is to run a short smoke test (e.g., 100 steps) before committing to a full training run.

Submission validation: Experiment 2 failed due to missing IDs in the submission file, a preventable error. Validation of submission completeness against the sample submission file should be a mandatory step before every upload.

Metric alignment: The disconnect between local ROUGE scores and leaderboard performance (most clearly seen in Experiment 4) highlights the importance of understanding evaluation metrics deeply. Local validation is useful for directional guidance but should not be treated as a perfect proxy for leaderboard performance.

8. Limitations

The data fraction used for fine-tuning varied across experiments (20% in Experiments 3-4, 10% in Experiment 5, 5% in Experiments 6-7) due to escalating compute constraints, which confounds direct comparison between experiments, most notably between Experiment 4 and Experiment 5 (epoch count vs. data fraction), and between Experiment 4 and Experiments 6-7 (model architecture vs. data fraction).

All fine-tuning experiments used a fraction of the available 28,000-example training set, limiting model exposure to the full topic and language distribution relative to the TF-IDF baseline's access to the complete set.

The local evaluation metrics (ROUGE) did not consistently align with the leaderboard metric across experiments, making it difficult to predict leaderboard impact from local validation alone.

9. Future Improvements

Several directions are identified as the most promising next steps, in priority order. First, the TF-IDF baseline (Experiment 1) is the strongest-performing approach found in this study and was not iterated on after the initial run; exploring word-level n-grams, wider character n-gram ranges, or a hybrid TF-IDF plus neural re-ranking approach is the highest-leverage remaining opportunity given the limited additional compute it requires. Second, a controlled rerun of Experiment 5 holding the data fraction fixed at 20% while varying only the epoch count would resolve the confound currently present and produce a clean answer to whether more training helps or causes overfitting. Third, training mT5-base at a full 20% data fraction (rather than the 5% forced by time constraints in Experiment 6) on a GPU with more available memory and time, such as an A100 via Colab Pro, would allow a fair, unconfounded comparison against mT5-small. Fourth, training the best-identified configuration on the full 28,000-example dataset, rather than a subsample, is expected to close a meaningful part of the gap to the TF-IDF baseline. Finally, exploring models specifically designed for African languages, such as CohereForAI's Aya-101, may yield further gains for the lowest-performing language subsets, particularly Luganda and Amharic.

References

Xue, L., Constant, N., Roberts, A., Kale, M., Al-Rfou, R., Siddhant, A., Barua, A., & Raffel, C. (2021). mT5: A massively multilingual pre-trained text-to-text transformer. *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics*, 483–498. <https://doi.org/10.18653/v1/2021.naacl-main.41>

Costa-jussà, M. R., Cross, J., Çelebi, O., Elbayad, M., Heafield, K., Heffernan, K., Kalbassi, E., Lam, J., Licht, D., Maillard, J., Sun, A., Wang, S., Wenzek, G., Youngblood, F., Akula, B., Barrault, L., Mejjia-Gonzalez, G., Hansanti, P., Hoffman, J., ... Wang, C. (2022). No language left behind: Scaling human-centered machine translation. *arXiv*. <https://arxiv.org/abs/2207.04672>

Lin, C.-Y. (2004). ROUGE: A package for automatic evaluation of summaries. Proceedings of the ACL Workshop on Text Summarization Branches Out, 74–81.

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., & Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140), 1–67.

Zindi Africa. (2026). Multilingual Health Question Answering in Low-Resource African Languages Challenge. <https://zindi.africa>